

E

Tentamen i Programmeringsteknik 1999-03-08, kl 8.00-13.00

Anvisningar

Hjälpmedel: Holm, Objektorienterad programmering och Java, samt Java-snabbreferens.

Skrivningen innehåller fyra uppgifter. Du bör behandla samtliga uppgifter.

Obs. Senast 3:e arbetsdagen efter tentamen anslås på anslagstavlan vilka som deltagit i tentamen men som, enligt institutionens noteringar, inte uppfyller tentamensvillkoren (alla 18 övningarna och de 3 inlämningsuppgifterna godkända). Deras skrivningar annulleras den 6:e arbetsdagen efter tentamen, såvida inte rättelse skett innan dess eller ansvarig lärare lämnat dispens.

Närmaste tentamenstillfällen i kursen:

Fredag 27 augusti 1999, kl 8-13, MA10

Mars 2000

Maj/juni 2000

Något om bedömningen

- Tänk på att du med dina lösningar vänder dig till en mänsklig rättare, inte till en dator.
- Du kan förutsätta att rättaren är väl insatt i uppgifterna.
- Rättaren fäster ingen större uppmärksamhet vid fel som en kompilator enkelt skulle upptäcka.
- Det är viktigt att du gör lämpliga och tydliga indragningar och att du använder vettiga variabelnamn så att din lösning går lätt att följa. En god handstil är ett plus.
- Tvekar du mellan att presentera en smart lösning eller en lösning som är enkel att följa, välj den enklare versionen.
- När det gäller betyget 3 så har det ingen betydelse om någon lösning är onödigt ineffektiv.

Uppgift 1

Två färdigskrivna klasser, en klass Card för ett kort i en kortlek och en klass CardDeck för en kortlek, har följande specifikationer:

```
specification Card {
    /** skapa ett kort med färgen color och valören value */
    Card(int color, int value);

    /** tag reda på kortets färg */
    int getColor();

    /** tag reda på kortets valör */
    int getValue();
}

specification CardDeck {
    /** skapa en kortlek med 4*13 = 52 kort */
    CardDeck();

    /** blanda kortleken */
    void shuffle();

    /** finns kort kvar i kortleken? */
    boolean cardsLeft();

    /** tag ett kort från kortleken */
    Card getCard();
}
```

Skriv ett programavsnitt där följande utförs:

- Bilda en blandad kortlek.
- Beräkna dels längden av den längsta kortsekvensen med samma färg dels valören av det sista kortet i denna sekvens. – Om det finns flera längsta sekvenser får vilken som helst av dem väljas.
- Skriv ut resultatet enligt följande exempel:

Längsta sekvensen har längden 5, sekvensens sista kort har valören 12

Uppgift 2

Implementera klasserna Card och CardDeck såsom de beskrivs i uppgift 1. Du skall alltså definiera lämpliga attribut och skriva konstruktörerna och operationerna i de båda klasserna.

En kortlek har 52 kort: fyra färger: spader (=1), hjärter (=2), ruter (=3), klöver (=4), vardera färgen med valörerna 1..13.

Använd en vektor med element numrerade 0..51 för kortleken.

Använd följande metod att blanda kortleken:

```
för k = 51 till 1:
    välj ett slumpstal s i intervallet [0..k]
    låt kort nr s byta plats med kort nr k
```

Uppgift 3

En deltagare i en tävling i armbrytning beskrivs av en klass med följande specifikation:

```
specification Player {
    /** skapa en deltagare med namnet name */
    Player(String name);

    /** tag reda på namnet */
    String getName();

    /** notera en ny vinst */
    void newWin();

    /** notera en ny förlust */
    void newLoss();

    /** tag reda på antalet vinster */
    int getNbrWins();

    /** tag reda på antalet förluster */
    int getNbrLosses();
}
```

100 deltagare är med i tävlingen. De finns i en vektor player som är attribut i en klass Table:

```
private Player[] player = new Player[100];
```

I vektorn är deltagarna sorterade så att den bäste finns i element nr 0 och den sämste i element nr 99. En deltagare är bättre än en annan deltagare om han har fler vinster än den andre, eller har lika många vinster men färre förluster. Ordningen mellan deltagare som har lika många vinster och lika många förluster har ingen betydelse.

Bland annat finns följande två metoder i klassen Table:

```
/** är deltagaren p1 bättre än deltagaren p2? */
public boolean better(Player p1, Player p2) {
    ...
}

/** notera att deltagaren p har vunnit ännu en match och tillse att
    han hamnar på rätt plats i vektorn player */
public void playerWins(Player p) {
    ...
}
```

Implementera metoderna better och playerWins. Observera att en spelare som vinner en match inte kan hamna på en sämre plats i vektorn.

Uppgift 4

Orden i en text finns i en lista words vars element beskrivs av klassen WordElement (listpaketet jlist används):

```
class WordElement extends Link {
    /** skapa ett element med ordet word */
    public WordElement(String word);

    /** tag reda på ordet */
    public String getWord();
}
```

Listan words är attribut i klassen Text:

```
class Text {
    private Head words;

    /** skapa en tom lista med plats för orden i en text */
    public Text();

    /** gruppera orden i texten i rader, se beskrivning nedan */
    public void makeLines(int maxWidth);
}
```

Man önskar skriva ut texten med en given största radlängd maxWidth och behöver därför operationen makeLines i klassen Text.

Operationen makeLines grupperar orden i listan i rader genom att infoga ett nytt wordElement med strängen "\n" (radbrytningstecken) efter varje "full rad". En "full rad" är en rad som innehåller så många ord som möjligt men vars längd (inklusive mellan-slagen mellan orden) inte överstiger maxWidth.

Implementera operationen makeLines, dvs infoga element med radbrytningstecken på avsedda platser i listan words.

Det får förutsättas att listan words inte är tom och att alla ordlängder är mindre än maxWidth.

Ledning: Längden av en sträng får man med operationen length() i klassen String.

Exempel:

Listan words innehåller
23 element med orden:

Vi
har
skaffat
nya
kunder
i
Tyskland
och
England.
Och
vår
nya
fabrik
i
Landskrona
kommer
att
öka
produktionen
under
1999,
säger
han.

makeLines med maxWidth=24
finner radindelningen:

Vi har skaffat nya
kunder i Tyskland och
England. Och vår nya
fabrik i Landskrona
kommer att öka
produktionen under 1999,
säger han.

6 nya element med "\n" har
infogats och listan words har
därmed 29 element.